1. Thuật toán Breadth-First Search (BFS)

1.1. Ý tưởng

BFS duyệt các trạng thái theo từng lớp độ sâu: mọi trạng thái cách trạng thái đầu k hành động được mở rộng trước các trạng thái cách $k+1$ hành động. Cài đặt dùng hàng đợi FIFO (collections.deque) và từ điển came_from làm tập đã thăm.

Bài kiểm tra đích được thực hiện ngay khi sinh trạng thái con, thay vì lúc lấy ra khỏi hàng đợi. Cách này giảm số nút phải mở rộng mà vẫn giữ được tính tối ưu, vì mọi hành động đều có chi phí 1.

1.2. Mã giả

Thuật toán 1: Breadth-First Search (BFS) cho Bloxorz

Đầu vào: màn chơi level, trạng thái đầu start
Đầu ra: dãy hành động ngắn nhất tới đích, hoặc vô nghiệm

```
nếu is_goal(start) thì trả về dãy rỗng
came_from[start] = None; queue = [start]; expanded = 0
trong khi queue không rỗng:
    state = queue.popleft(); expanded += 1
    với mỗi (action, nxt) trong get_successors(state, level):
        nếu nxt đã có trong came_from thì bỏ qua
        came_from[nxt] = (state, action)
        nếu is_goal(nxt) thì trả về đường đi dựng lại từ came_from
        queue.append(nxt)
trả về vô nghiệm (hàng đợi cạn)
```

1.3. Cài đặt

Vòng lặp chính trong solvers/BFS.py:

```python
came_from = {start: None}
queue = deque([start])
expanded = 0

while queue:
    state = queue.popleft()
    expanded += 1
    for action, nxt in get_successors(state, level):
        if nxt in came_from:
            continue
        came_from[nxt] = (state, action)
        if is_goal(nxt, level):
            return make_result(reconstruct_path(came_from, nxt), expanded)
        queue.append(nxt)
```

Nếu trạng thái đầu đã là đích, solver trả về dãy rỗng. Nếu hàng đợi cạn mà chưa gặp đích, solver trả về path=None, tức màn chơi vô nghiệm. Đường đi được dựng lại bằng reconstruct_path trong solvers/common.py, dùng chung với các solver khác.

1.4. Tính chất

- **Đầy đủ**: nếu có lời giải thì BFS tìm được, vì không gian trạng thái hữu hạn.
- **Tối ưu**: vì mọi hành động có chi phí 1, lời giải tìm được có số hành động ít nhất.

- **Độ phức tạp**: với hệ số nhánh $b \leq 4$ (hoặc $b \leq 5$ khi khối bị tách) và độ sâu lời giải d, thời gian và bộ nhớ đều ở mức $O\left(b^d\right)$. Nhờ tập trạng thái đã thăm, cả hai còn bị chặn trên bởi số trạng thái đạt được $|S|$.
- **Nhược điểm chính**: phải giữ toàn bộ biên tìm kiếm và mọi trạng thái đã thăm trong bộ nhớ, nên bộ nhớ tăng nhanh theo kích thước không gian trạng thái.

Ghi chú

Số nút mở rộng thực tế (xem phần thực nghiệm) nhỏ hơn rất nhiều so với b^d, vì nhiều chuỗi hành động khác nhau dẫn tới cùng một trạng thái và bị loại bởi came_from.

2. Thực nghiệm BFS

2.1. Thiết lập

BFS chạy trên cùng bộ màn chơi do load_all_levels() nạp, với trạng thái đầu GameState(level.start, level.initial_bridge_mask). Các số liệu được đo như sau:

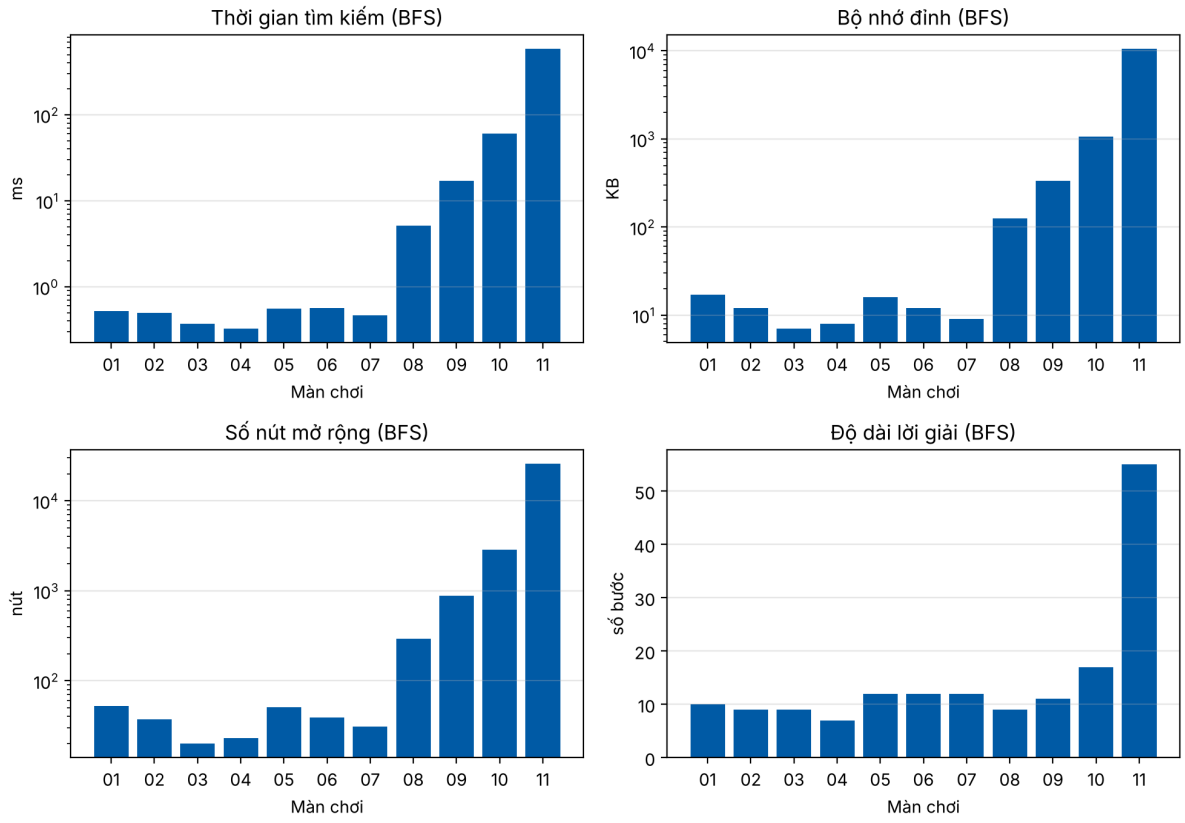
- **Thời gian tìm kiếm**: time.perf_counter(), lấy trung bình và độ lệch chuẩn qua nhiều lần chạy, **không** bật tracemalloc (vì tracemalloc làm chương trình chậm đi đáng kể).
- **Bộ nhớ đỉnh**: tracemalloc.get_traced_memory(), đo ở một lần chạy riêng (hàm run_solver trong solvers/common.py).
- **Số nút mở rộng**: số trạng thái được lấy ra khỏi hàng đợi và sinh trạng thái con. Nút đích được phát hiện lúc sinh nên không được tính là đã mở rộng.
- **Số bước di chuyển**: độ dài đường đi trả về; hành động SPACE tính là một bước.

(CẦN BỔ SUNG) Mô tả các nhóm màn chơi (cơ bản, có ô dễ vỡ, có công tắc/cầu, có công tắc tách) và màn nào thuộc nhóm nào; cấu hình máy (CPU, RAM, hệ điều hành, phiên bản Python); số lần lặp đã dùng khi đo thời gian.

2.2. Kết quả

Bảng 1: Kết quả của BFS trên bộ màn chơi thử nghiệm.

Màn	Tìm được	Thời gian TB (ms)	Độ lệch chuẩn (ms)	Bộ nhớ đỉnh (KB)	Nút mở rộng	Độ dài lời giải
level_01	Có	0.525	0.046	17	52	10
level_02	Có	0.5	0.131	12	37	9
level_03	Có	0.373	0.028	7	20	9
level_04	Có	0.325	0.085	8	23	7
level_05	Có	0.552	0.138	16	51	12
level_06	Có	0.564	0.116	12	39	12
level_07	Có	0.463	0.071	9	31	12
level_08	Có	5.143	0.633	125	292	9
level_09	Có	17.156	0.783	333	887	11
level_10	Có	60.742	6.993	1060	2864	17
level_11	Có	584.956	33.589	10552	25815	55



Hình 1: Thời gian, bộ nhớ đỉnh, số nút mở rộng và độ dài lời giải của BFS theo từng màn (dùng thang log khi giá trị trải rộng).

2.3. Nhận xét

BFS giải được cả 11 màn và luôn cho lời giải ngắn nhất (từ 7 đến 55 bước), đúng với lý thuyết khi mọi hành động có chi phí 1.

Ở các màn nhỏ (level_01 đến level_07), BFS mở rộng từ 20 đến 52 nút, mất khoảng 0,3 đến 0,6 ms và 7 đến 17 KB bộ nhớ. Ở mức này, bộ nhớ chủ yếu phản ánh chi phí cố định của Python nên chênh lệch giữa các màn không có nhiều ý nghĩa.

Tài nguyên tăng nhanh theo độ khó. level_08, level_09 và level_10 lần lượt mở rộng 292, 887 và 2864 nút (5,1; 17,2 và 60,7 ms; 125, 333 và 1060 KB). level_11 là màn lớn nhất: BFS mở rộng 25815 nút, mất khoảng 585 ms và 10552 KB, với lời giải 55 bước.

Dù độ sâu lời giải của level_11 là 55, số nút mở rộng vẫn nhỏ hơn rất nhiều so với b^d, cho thấy tập trạng thái đã thăm loại bỏ được phần lớn trạng thái trùng. Nhược điểm của BFS là phải giữ toàn bộ biên tìm kiếm trong bộ nhớ nên chi phí tăng theo kích thước không gian trạng thái. So sánh với DFS được trình bày trong phần DFS.

(CẦN BỔ SUNG) Nhận xét thêm theo nhóm màn (ví dụ màn có công tắc tách có làm số nút tăng đột biến không) sau khi đã xác định màn nào thuộc nhóm nào.

2.4. Kết luận

BFS phù hợp khi cần lời giải ít bước nhất và không gian trạng thái ở mức vừa phải. Điểm mạnh là tính tối ưu và hiệu năng ổn định; điểm yếu là bộ nhớ tăng nhanh ở các màn lớn (10552 KB ở level_11).